DATA Structure and Algorithms

(BCO 002B)

UNIT 3

Department of Computer Science & Engineering
JECRC University

Syllabus

BCO002B	DATA STURCTURES AND ALGORITHM	4-0-0 [4]
UNIT 1	Introduction: Notions of data type, abstract data type and data structures. Importance of algorithms and data structures in programming. Notion of Complexity covering time complexity, space complexity, Worst case complexity & Average case complexity. BigOh Notation, Omega notation,Theta notation. Examples of simple algorithms and illustration of their complexity. Sorting- Bubble sort, selection sort, insertion sort, Quick sort; Heap sort; Merge sort; Analysis of the sorting methods. Selecting the top k elements. Lower bound on sorting.	
UNIT 2	Stack ADT, Infix Notation, Prefix Notation and Postfix Notation. Evaluation of Postfix Expression, conversion of Infix to Prefix and Postfix Iteration and Recursion- Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion.	
UNIT 3	List ADT. Implementation of lists using arrays and pointers. Stack ADT. Queue ADT. Implementation of stacks and queues. Dictionaries, Hash tables: open tables and closed tables. Searching technique- Binary search and linear search, link list- single link list, double link list, Insertion and deletion in link list.	
UNIT 4	Binary Trees- Definition and traversals: preorder, post order, in order. Common types and properties of binary trees. Binary search trees: insertion and deletion in binary search tree worst case analysis and average case analysis. AVL trees. Priority Queues -Binary heaps: insert and delete min operations and analysis.	
UNIT 5	Graph: Basic definitions, Directed Graphs- Data structures for graph representation. Shortest path algorithms: Dijkstra (greedy algorithm) and Operations on graph, Worshall's algorithm , Depth first search and Breadth-first search. Directed acyclic graphs. Undirected Graphs, Minimal spanning trees and algorithms (Prims and Kruskal) and implementation. Application to the travelling salesman problem.	

Course Outcome	Program Outcome												Program Specifice Outcome		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	H												M		
CO2			H		M								M		
CO3		H							M			L		H	
CO4	H	M											L		L
CO5		M		H											L

Course Objective

CO1:Show the understanding of asymptotic analysis of algorithm
CO2: Understand the concepts of Stacks and Queues with their application
CO3: Understand with utilization concepts of Linked List with their application
CO4:Use comprehensive knowledge Trees in problem solving.
CO5:Use comprehensive knowledge of Graph.

Books & References

Text Books:

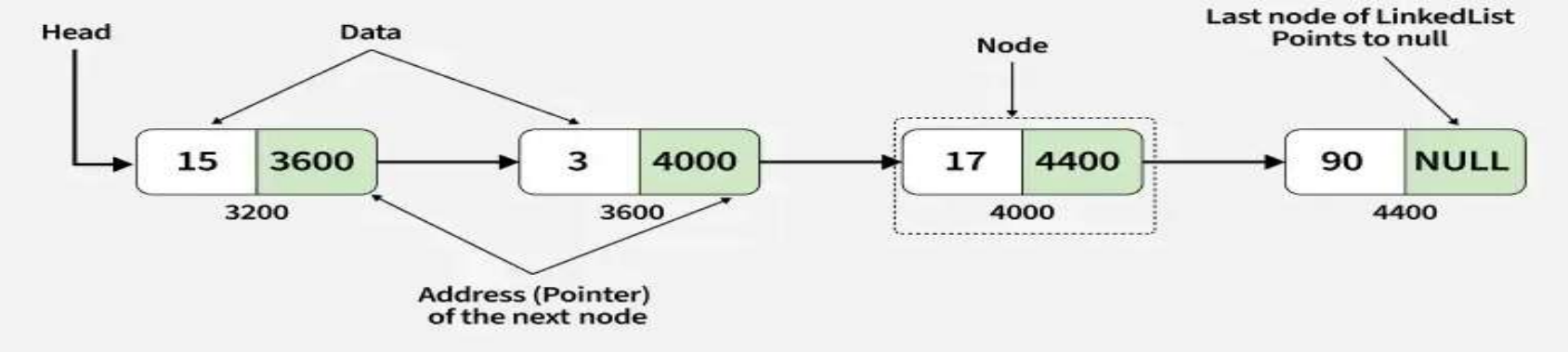
1. Data Structures and Algorithms by Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft , Addison-Wesley Series (1983)

Reference Books:

T.H. Cormen, C.E. Leiserson, and R.L. Rivest. Introduction to Algorithms.The MIT Press and McGraw-Hill Book Company, Cambridge, Massacusetts, 1990 (Available in Indian Edition).
Steven S. Skiena. The Algorithm Design Manual.Springer, Second Edition, 2008.
Data Structures and Algorithm Analysis in Java (3rd Edition) by Mark Allen Weiss, Addison Wesley(2011)

Linked List

- A linked list is a fundamental data structure in computer science. It mainly allows efficient insertion and deletion operations compared to arrays. Like arrays, it is also used to implement other data structures like stack, queue and deque.



Linked List

- **Data Structure:** Non-contiguous
- **Memory Allocation:** Typically allocated one by one to individual elements
- **Insertion/Deletion:** Efficient
- **Access:** Sequential

Operations

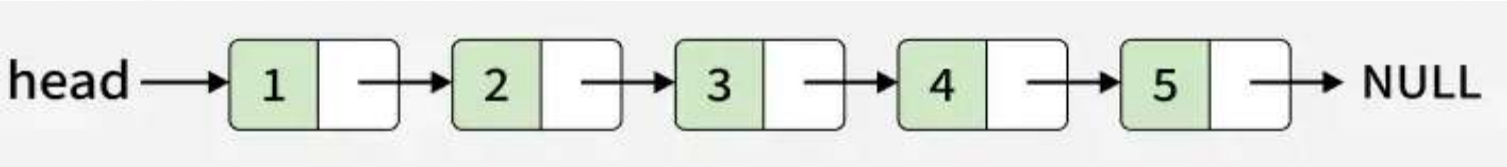
- Length of Linked List
- Print Linked List
- Search in a Linked List
- Linked List Insertion
- Deleting a given key
- Deleting at given position
- Delete a Linked List
- Nth Node from Start
- Nth Node from End
- Size of Doubly Linked List

Types of Linked List

- Singly Linked List
- Doubly Linked List
- Circular Linked List

Singly Linked List

- A singly linked list is a fundamental data structure, it consists of nodes where each node contains a data field and a reference to the next node in the linked list. The next of the last node is null, indicating the end of the list. Linked Lists support efficient insertion and deletion operations.



Understanding Node Structure

```
class Node {

public:
    // Data part of the node
    int data;

    // Pointer to the next node in the list
    Node* next;

    // Constructor to initialize the node with data
    Node(int data) {
        this->data = data;
        this->next = NULL;
    }
};
```

Creating an Example Linked List

Create the first node

Allocate memory for the first node and Store data in it.

Mark this node as head.

Create the second node

Allocate memory for the second node and Store data in it.

Link the first node's next to this new node.

Create the third node

Allocate memory for the third node and Store data in it.

Link the second node's next to this node.

Set its next to NULL to ensure that the next of the last is NULL.

Singly linked list node structure

```
// singly linked list node structure
class Node {
public:
    int data;
    Node* next;

    // constructor to initialize a new node with data
    Node(int new_data) {
        this->data = new_data;
        this->next = nullptr;
    }
};

int main() {
```

•

```
// Create the first node (head of the list)
    Node* head = new Node(10);
    // Link the second node
    head->next = new Node(20);
    // Link the third node
    head->next->next = new Node(30);
    // Link the fourth node
    head->next->next->next = new Node(40);
    // printing linked list
    while (head != nullptr) {
        cout << head->data << " ";
        head = head->next;
    }
}
```

Common Operation in Linked List

A linked list supports several operations. Here are the most common ones:

Traversal : Traversing singly linked list

Insertion : At the beginning, At the end and At a specific position

Deletion : From beginning, From end and From a specific position

Searching : Find whether a given key exists in the list

Updating (Modification) : Modify contents of linked list.

Reversal : Reverse the linked list and make the last node as new head.

Advantage

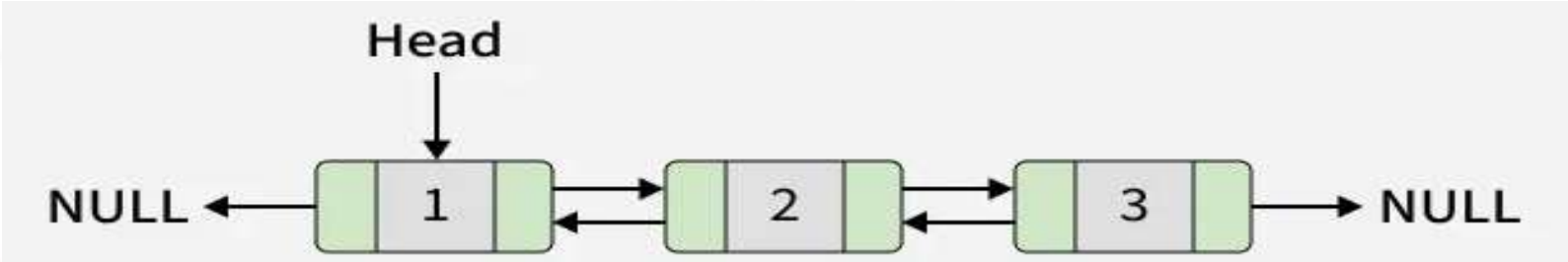
- Dynamic size (no fixed limit like arrays)
- Efficient insertion and deletion at beginning and end. We also
- have insertion at the middle efficient if we have reference or pointer to the node after which we need to insert.
- Can implement complex data structures like stack, queue, graph

Disadvantage

- Extra memory required for storing pointers
- No direct/random access (need traversal)
- Cache unfriendly (not stored in contiguous memory)

Doubly Linked List

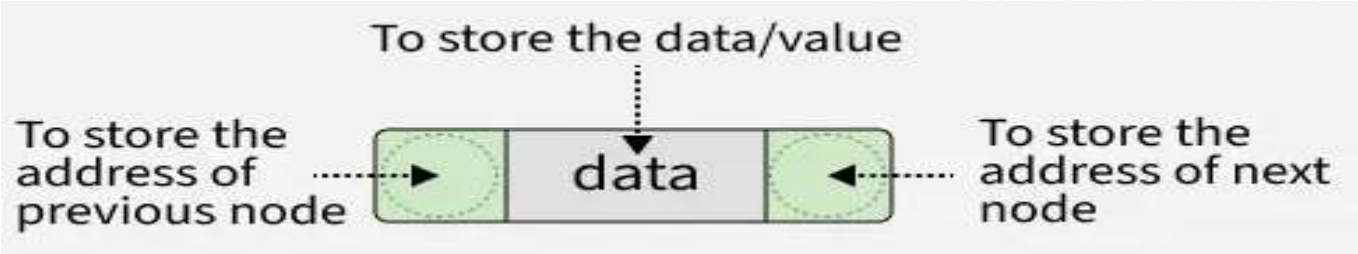
- A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.



Representation of Doubly Linked List

In a data structure, a doubly linked list is represented using nodes that have three fields:

- Data
- A pointer to the next node (next)
- A pointer to the previous node (prev)



Node Definition

```
#include <iostream>
using namespace std;
class Node {
public:
    // To store the Value or data
    int data;
    // Pointer to point the Previous Element
    Node* prev;
    // Pointer to point the Next Element
    Node* next;
    // Constructor
    Node(int d) {
        data = d;
        prev = nullptr;
        next = nullptr;
    }
};
```

Creating a Doubly Linked List

Create the head node.

•Allocate a node and set head to it. Its prev and next should be null/None.

Create the next node and link it to head.

•head.next = new Node(value2)

•head.next.prev = head

Create further nodes the same way.

•For the third node:
=> head.next.next = new Node(value3)
=> head.next.next.prev = head.next

•Repeat until you have the required nodes.

Ensure the tail's next is null.
The last node you created must have next == null

Set / keep track of head (and optionally tail).
Use head to access the list from the front. Keeping a tail pointer simplifies appends.

Common Operation in Doubly Linked List

- Traversal : Display Linked List Elements
- Insertion : At the Beginning, At the End and At the specific position
- Deletion : From the Beginning, From End and From a Specific Position

Advantages of Doubly Linked List

- Bidirectional Traversal - You can traverse forward (using next) as well as backward (using prev).
- Efficient Deletion - Given a pointer to a node, you can delete it in O(1) time (no need to traverse from the head), since you can update both prev and next.
- Insertion at Both Ends - Insertion at head or tail is efficient because you can update both directions easily.
- Easy to Implement Deque / Navigation Features - Useful for undo/redo, browser history, and music playlist navigation, where both forward and backward movement is needed.

Disadvantages of Doubly Linked List

- Extra Memory Per Node - Each node requires an additional pointer (prev), making DLL more memory-consuming than singly linked list.
- More Complex Implementation - Both prev and next must be handled carefully during insertion and deletion, which increases chances of errors (broken links, null pointer issues)
- Slower Operations Due to Overhead - Extra pointer manipulations during insertion/deletion cause slightly more overhead compared to singly linked list.
- Not Cache-Friendly - Like singly linked list, nodes are scattered in memory, so traversals may be slower compared to arrays due to poor locality of reference.

Queue

- A **Queue** is a **linear data structure** that follows the **FIFO (First In First Out)** principle.
- Think of it like a real-world **line of people waiting at a ticket counter**.

Characteristics of Queue

- Linear structure
- Operations happen at **two ends**:
  - **Front** → Deletion
  - **Rear** → Insertion
- Maintains **order of elements**
- Used in **sequential processing**

Real-Life Examples

- Printer job scheduling
- Call center waiting system
- Traffic lights
- Ticket booking system

Basic Queue Operations

- **Enqueue:** Insert element at the rear
- **Dequeue:** Remove element from the front
- **Peek/Front:** Get front element
- **isEmpty / isFull:** Check status

- **Enqueue Operation:**

```
    if rear == MAX - 1:
    print("Queue Overflow")
else:
    rear = rear + 1
    queue[rear] = item
```

Cont……

Dequeue Operation

```
if front > rear:
    print("Queue Underflow")
else:
    item = queue[front]
    front = front + 1
```

Peek Operation

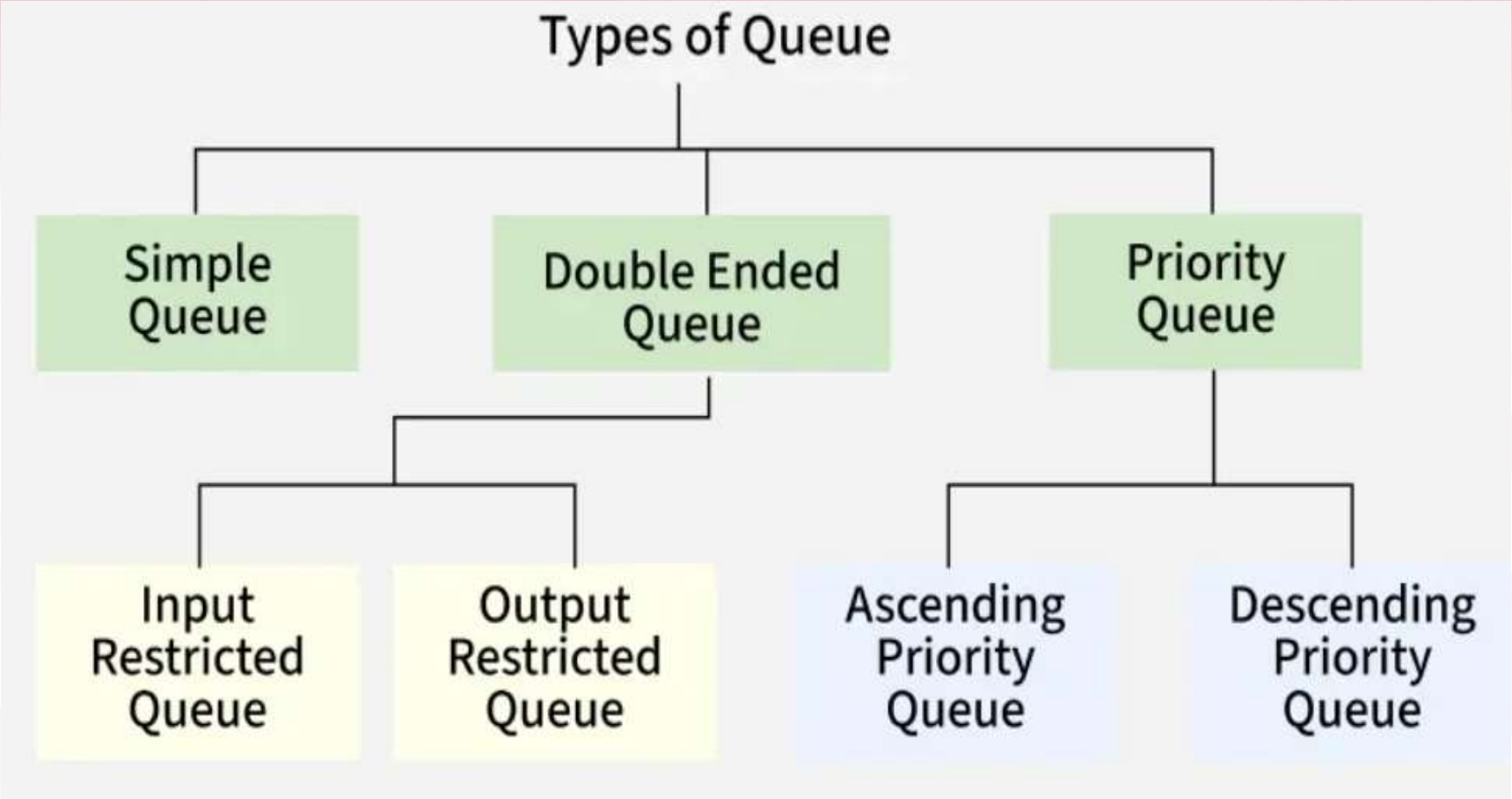
Purpose: Returns the front element without removing it.

Example: Queue = [20, 30, 40] → Peek = 20

Queue – Linked List Implementation

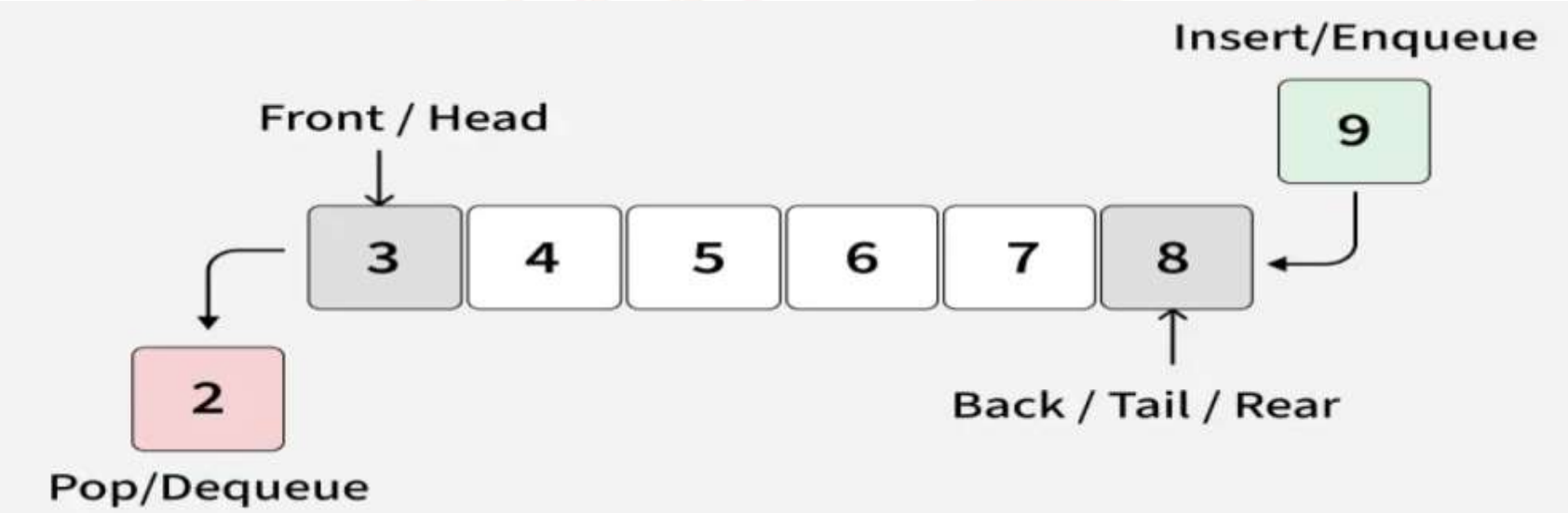
```
public:
    int data;
    Node* next;
    Node(int new_data) {
        data = new_data;
        next = nullptr;}
  };
class myQueue {
private:
    Node* front;
    Node* rear;

public:
    myQueue() {
        front = rear = nullptr;
    }
};
```

Types of Queues



Simple Queue

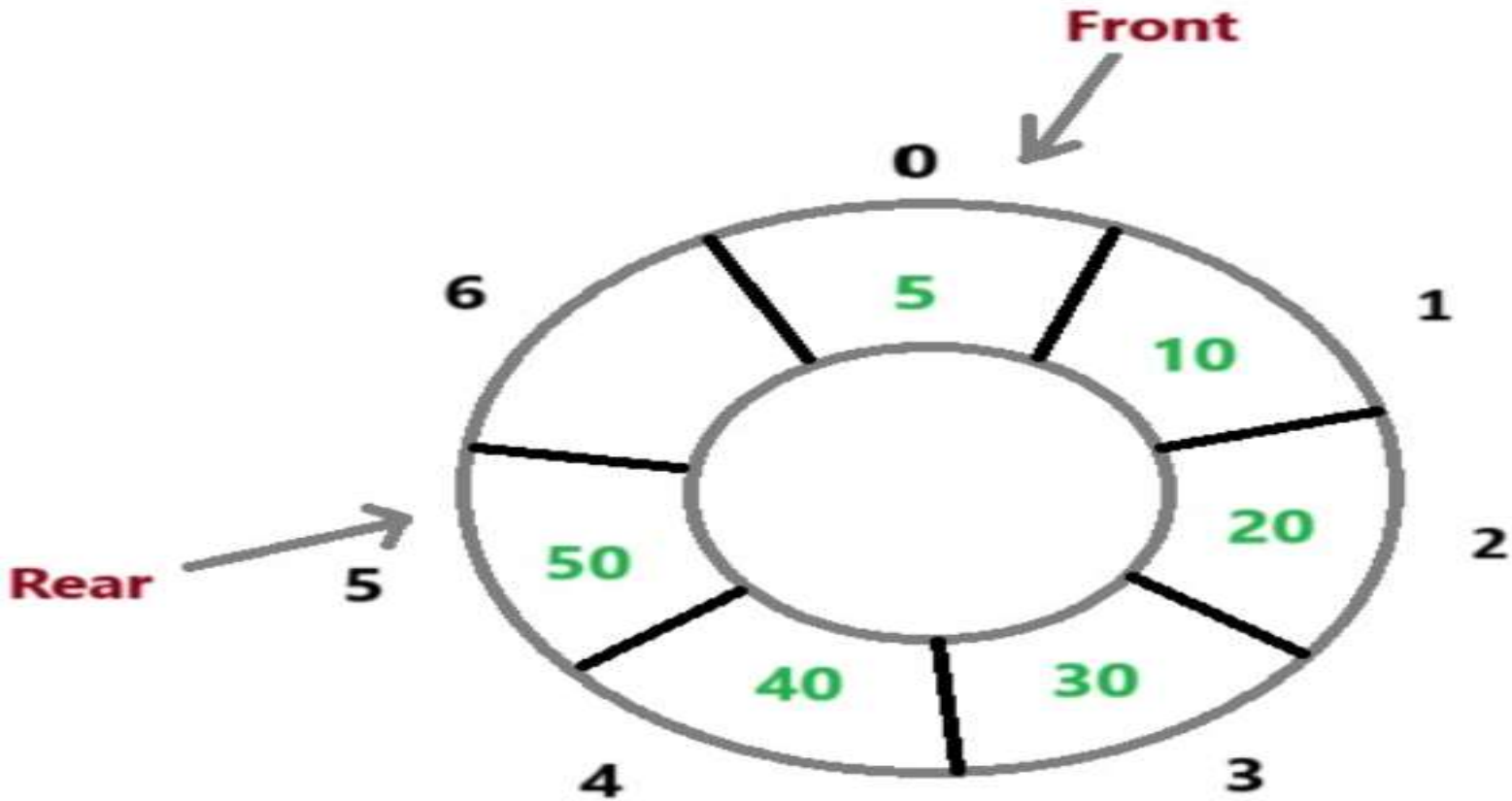
Basic FIFO queue.
Example: Queue in a movie



Circular Queue

- A circular queue is a type of queue in which the last position is connected back to the first position to make a circle. It is also known as a Ring Buffer. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using a circular queue, we can use the space to insert elements.
- It is a linear data structure that follows the FIFO mechanism. The circular queue is a more efficient way to implement a queue in a fixed size array. In a circular queue, the last element points to the first element making a circular link.

Representation of Circular Queue



Operations on Circular Queue

- **Enqueue:** Insert an element into the circular queue.
- **Dequeue:** Delete an element from the circular queue.
- **Front:** Get the front element from the circular queue.
- **Rear:** Get the last element from the circular queue.

Time Complexity of Circular Queue

- **Enqueue Operation:** The time complexity of the enQueue operation is O(1) as we can insert an element at the rear end of the queue in constant time.
- **Dequeue Operation:** The time complexity of the deQueue operation is O(1) as we can delete an element from the front end of the queue in constant time.
- **Front Operation:** The time complexity of the front operation is O(1) as we can get the front element of the queue in constant time.
- **Rear Operation:** The time complexity of the rear operation is O(1) as we can get the rear element of the queue in constant time.

Applications of Circular Queue

- **Memory Management:** Circular queues are used in memory management to manage the memory efficiently. It is used to allocate memory to the processes when they are in need of memory.
- **Buffer Management:** These queues are also useful in buffer management. Consider a situation where data is produced at a faster rate than it is consumed. In such cases, a circular queue is used to store the data temporarily.
- **Operating System:** Suppose your system has a lot of processes to execute. In such cases, for the better management of processes, the operating system uses a circular queue to allocate the CPU to the processes.
- **Traffic Management:** Traffic singals are controlled by the circular queue. Let's say there are three signals, red, yellow, and green. The signals are in a circular queue. When the green signal is on, the next signal will be yellow, and then red. This process will continue in a circular manner.
- **Multimedia Player:** When we play songs in a multimedia player, the songs are played in a circular manner. Once the last song is played, the first song will be played next. This is done using a circular queue.

Priority Queue

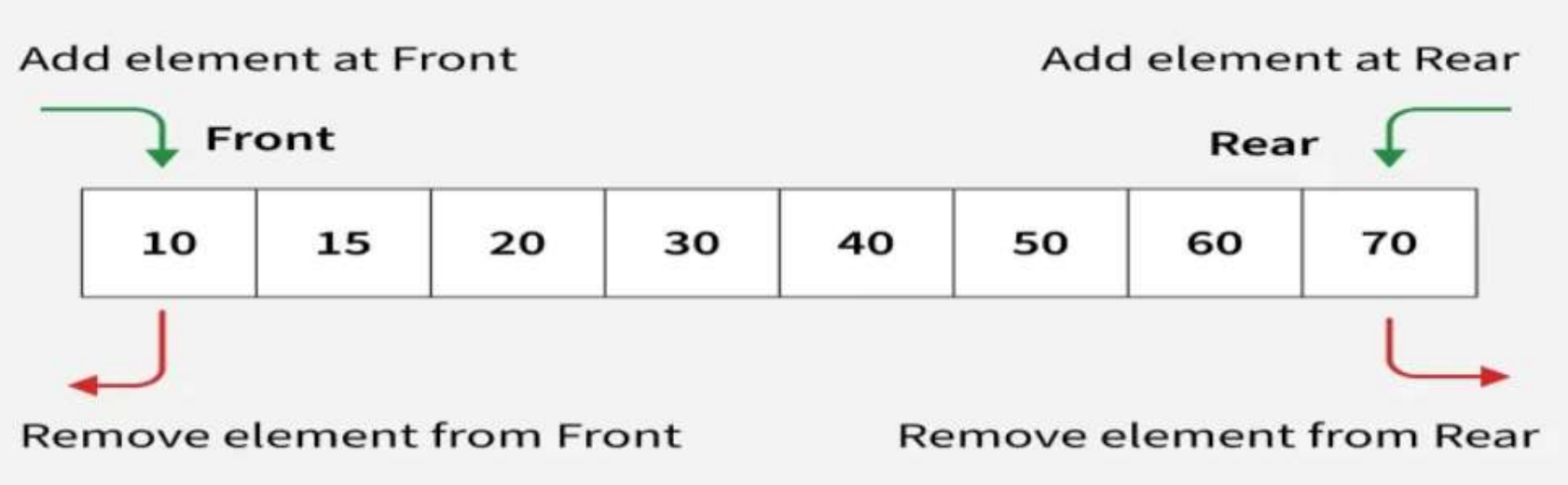
Priority Queue is more specialized data structure than Queue. Like ordinary queue, priority queue has same method but with a major difference. In Priority queue items are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa. So we're assigned priority to item based on its key value. Lower the value, higher the priority.

Basic Operations

- **insert / enqueue** − add an item to the rear of the queue.
- **remove / dequeue** − remove an item from the front of the queue.
- **Peek** − get the element at front of the queue.
- **isFull** − check if queue is full.
- **isEmpty** − check if queue is empty.

Deque

Deque or Double Ended Queue is a generalized version of Queue data structure that allows insert and delete at both ends.



- Deque can act as both Stack and Queue
- It is useful in many problems where we need to have a subset of all operations also like insert/remove at front and insert/remove at the end.
- It is typically implemented either using a doubly linked list or circular array.

Types of Deques

- **Input-Restricted Deque:**

In this type of Deque, insertion is allowed only at one end, typically the rear.

Deletion, however, can be performed from both ends: (front and rear).

This structure functions like a queue for insertion but offers more flexibility for removal.

- **Output-Restricted Deque:**

In this type of Deque, deletion is allowed only at one end, typically the front.

Insertion, however, can be performed from both ends: (front and rear).

This structure functions like a queue for deletion but offers more flexibility for adding elements.

Common Operations on Deque

1. **push_front(value) :** This operation adds an element at the beginning of the deque, shifting other elements to the right if necessary.
Example: If the deque is [2, 3, 4], after push_front(1), the deque becomes [1, 2, 3, 4].
2. **push_back(value):** This operation adds an element at the end of the deque without needing to shift other elements.
Example: If the deque is [1, 2, 3], after push_back(4), the deque becomes [1, 2, 3, 4].
3. **pop_front():** This operation removes the first element of the deque, shifting the remaining elements to the left.
Example: If the deque is [1, 2, 3], after pop_front(), the deque becomes [2, 3].
4. **pop_back():** This operation removes the last element from the deque without shifting any elements.
Example: If the deque is [1, 2, 3], after pop_back(), the deque becomes [1, 2].
5. **front():** This operation gives the value of the first element of the deque, which is the element at the front.
Example: If the deque is [1, 2, 3], front() will return 1.

Example: If the deque is [1, 2, 3], back() will return 3.

7. **empty():** This operation checks if there are any elements in the deque. It returns true if the deque is empty, and false otherwise. **Example**: If the deque is [1, 2, 3], empty() will return false. If the deque is [], empty() will return true.

8. **size():** This operation returns the number of elements present in the deque without modifying it.

Cont…..

Dictionaries

When presented with a key the dictionary will return the associated value.

A dictionary is also called a hash, a map, a hashmap in different programming languages.

The keys in a dictionary must be simple types (such as integers or strings) while the values can be of any type.

the values

- results =

{ 'Detra' : 17,
'Nova' : 84,
'Charlie' : 22,
'Henry' : 75,
'Roxanne' : 92,
'Elsa' : 29 }

- Instead of using the numerical index of the data we can use the dictionary names to return values
- >>> results

['Nova']
84

- >>> results

['Elsa']
29

missing key may give a default value or throw an exception)

operations

– insert or update a value (typically, if the key does not exist in the dictionary, the key-value pair is inserted; if the key already exists, its corresponding value is overwritten with the new one)

– remove a key-value pair – test for existence of a key

HASHING

JECRC UNIVERSITY
BUILD YOUR WORLD

Introduction

- Hashing is a technique that is ***used*** to ***uniquely identify a specific object*** from a ***group of similar objects***.
- Some examples of how hashing is used in our lives include:
  - In universities, each student is assigned a unique roll number that can be used to retrieve information about them.
  - In libraries, each book is assigned a unique number that can be used to determine information about the book, such as its exact position in the library or the users it has been issued to etc.
- In both these examples the students and books were hashed to a ***unique number***.

- In ***hashing***, ***large keys*** are ***converted*** into ***small keys*** by using ***hash functions***.
- The values are then ***stored in a data structure*** called ***hash table***.
- The ***idea*** of hashing is to ***distribute entries*** (key/value pairs) ***uniformly across an array***.
- Each element is assigned a key (converted key).
- By using that key you can access the element in **O(1)** time. Using the key, the ***algorithm*** (hash function) computes an index that suggests where an entry can be ***found*** or ***inserted***.

- Hashing is implemented in two steps:
  - An ***element is converted into an integer*** by using a ***hash function***. This element can be used as an ***index to store the original element***, which falls into the hash table.
  - The element is stored in the hash table where it can be quickly retrieved using ***hashed key***.
  - hash = hashfunc(key)
    index = hash % array_size

Hash function

- A ***hash function*** is any function that can be ***used*** to ***map a data*** set of an ***arbitrary size*** to a ***data set of a fixed size***, which falls into the hash table.
- The values returned by a hash function are called hash values, hash codes, hash sums, or simply hashes.

Hash Table

- A hash table is a data structure that is used to store keys/value pairs. It uses a hash function to compute an index into an array in which an element will be inserted or searched.
- By using a good hash function, hashing can work well. Under reasonable assumptions, the average time required to search for an element in a hash table is **O(1)**.

Issues with Hashing

Design hash functions such that collisions are minimized.

But avoiding collisions is impossible.

Design collision-resolution techniques.

Search will cost $\Theta(n)$ time in the worst case.

However, all operations can be made to have an expected complexity of $\Theta(1)$.

Collision resolution techniques

- If x_1 and x_2 are two different keys, it is possible that $h(x_1) = h(x_2)$. This is called a collision. Collision resolution is the most important issue in hash table implementations.
- Choosing a hash function that minimizes the number of collisions and also hashes uniformly is another critical issue.
  - *Separate chaining (open hashing)*
  - *Linear probing (open addressing or closed hashing)*
  - *Quadratic Probing*
  - *Double hashing*